

Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

University of Minho
School of Engineering

Data Security

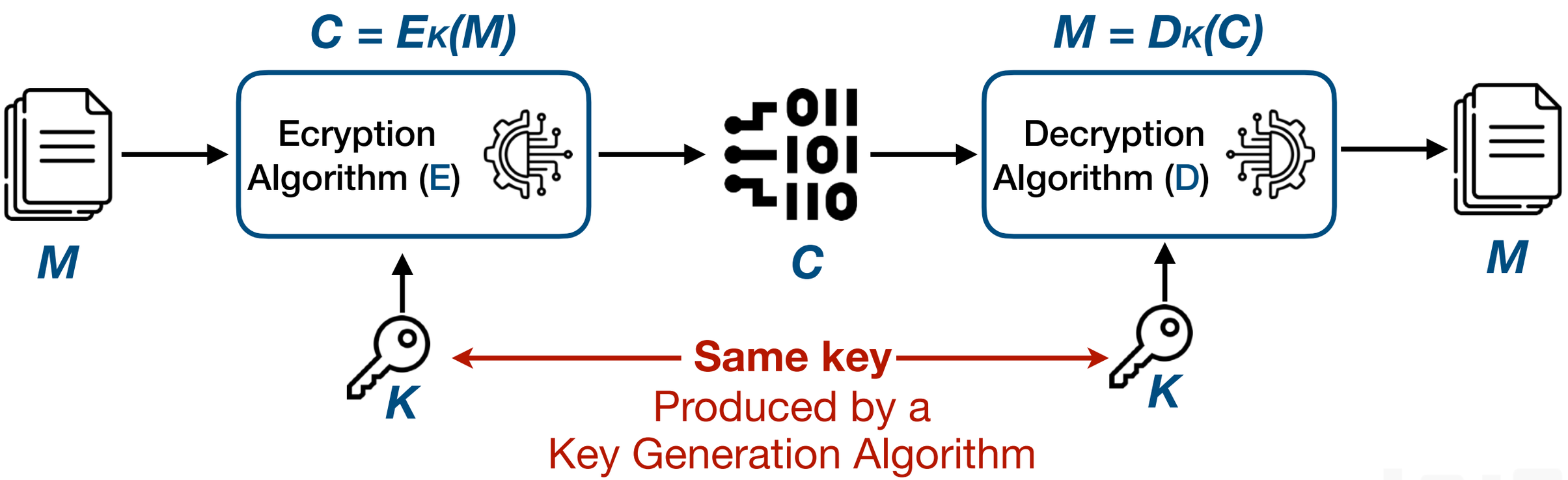
Objectives

- Understand how stream ciphers work
- Comprehend why their security is dependent on the randomness and unpredictability of the used keystream
- Understand the role of random number generators for stream ciphers
- Understand the concepts and requirements for pseudorandom number generators

Data Security

Recap

Symmetric cryptography

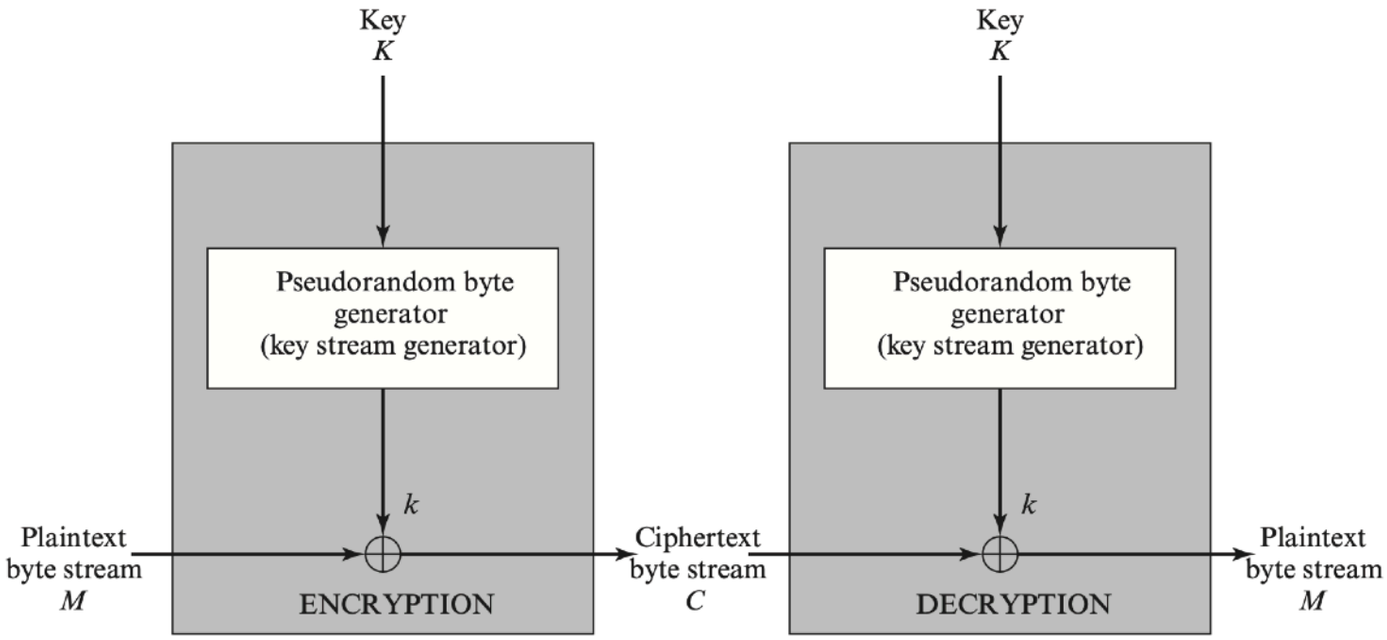


- **M** - Plaintext
- **E** - Encryption algorithm
- **K** - Secret key
- **C** - Ciphertext
- **D** - Decryption algorithm

Stream Ciphers

The cryptographic algorithm processes the input elements (bit, byte, or character) of the plaintext continuously, producing output one element at a time.

- They typically use the key as input to a pseudorandom bit generator that produces a stream of elements (*e.g.,* bits or bytes) that are apparently random.
- The output of the generator, called `keystream`, is combined one element at a time with the plaintext stream, *e.g.,* using `XOR` operation.
- They are similar to the *One-Time Pad*. However, stream ciphers use a pseudorandom number stream.



Source: Stallings, W. & Brown, L. Cryptography and Network Security: Principles and Practice

Stream Ciphers

The cryptographic algorithm processes the input elements (bit, byte, or character) of the plaintext continuously, producing output one element at a time.

Encryption process

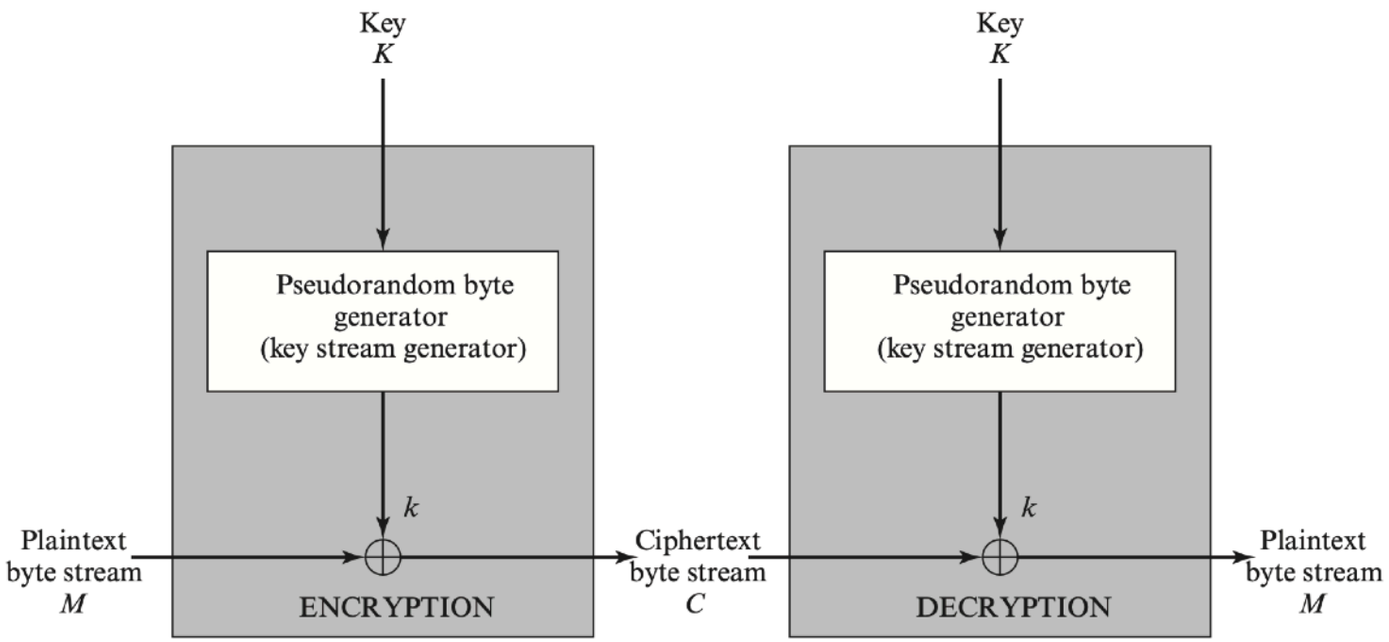
```
  1100110011001100   Plaintext
⊕ 0110110001101100   Keystream
  ----------------
  1010000010100000   Ciphertext
```

Decryption process

```
  1010000010100000   Ciphertext
⊕ 0110110001101100   Keystream
  ----------------
  1100110011001100   Plaintext
```

Stream Ciphers

- Synchronization
  - *Synchronous*: the key sequence is independent of the message
  - *Self-synchronizing*: with the ability to recover synchronization even when bits of the cryptogram are lost.



Stream Ciphers

One-Time Pad

An unconditionally secure cipher, that:

- **Uses a key as long as the message (*i.e.*, plaintext);**
- **The key is randomly generated;**
- **Each key is used to encrypt and decrypt a single message.**

It produces random output that bears no statistical relationship to the plaintext.

It usually operates over a binary alphabet using *XOR* operations.

$$C_i = p_i \oplus k_i \qquad p_i = C_i \oplus k_i$$

In practice, its security is entirely due to the randomness of the key. If the stream of characters (or bits) in the key is truly random, then so are the characters (or bits) in the ciphertext.

Stream Ciphers

One-Time Pad

Despite offering perfect secrecy, the one-time pad has two fundamental drawbacks:

- Generating large numbers of random and unpredictable keys is a practical problem
- Key distribution is an even more challenging problem since each message to be sent requires a new key of the same size that must be shared with the receiver

```
Message 1 | keystream 1
Message 2 | keystream 2
Message 3 | keystream 3
          .
          .
          .
Message n | keystream n
```



Stream Ciphers

One-Time Pad

What happens when encrypting multiple messages with the same key?

$C_1 = m_1 \oplus k_a$

$C_2 = m_2 \oplus k_b$

If $k_a == k_b$

$C_1 \oplus C_2 == m_1 \oplus m_2$

This violates the definition of security, where, regardless of any prior information an attacker has about the plaintext, the ciphertext should reveal no additional information about the plaintext.

- In practice
  - It allows frequency-based cryptanalysis
  - The higher the number of ciphertexts, the easier the cryptanalysis

See more here

Stream Ciphers

Random Bit Generation

The use of Random Numbers (binary sequences)

Random binary numbers are used in multiple security algorithms and cryptographic protocols:

- Key distribution
- Bidirectional authentication
- Session key generation
- Generation of keys for the RSA public-key encryption algorithm
- Generation of a bit stream for symmetric stream encryption

Stream Ciphers

Random Bit Generation

Principles and Requirements for generators

Randomness

The sequence of numbers has to be random in some well-defined statistical sense. Two criteria are used to validate that:

- **Uniform distribution**: The distribution of bits (zeros and ones) in the sequence should be uniform.
- **Independence**: No one subsequence in the sequence can be inferred from the others.

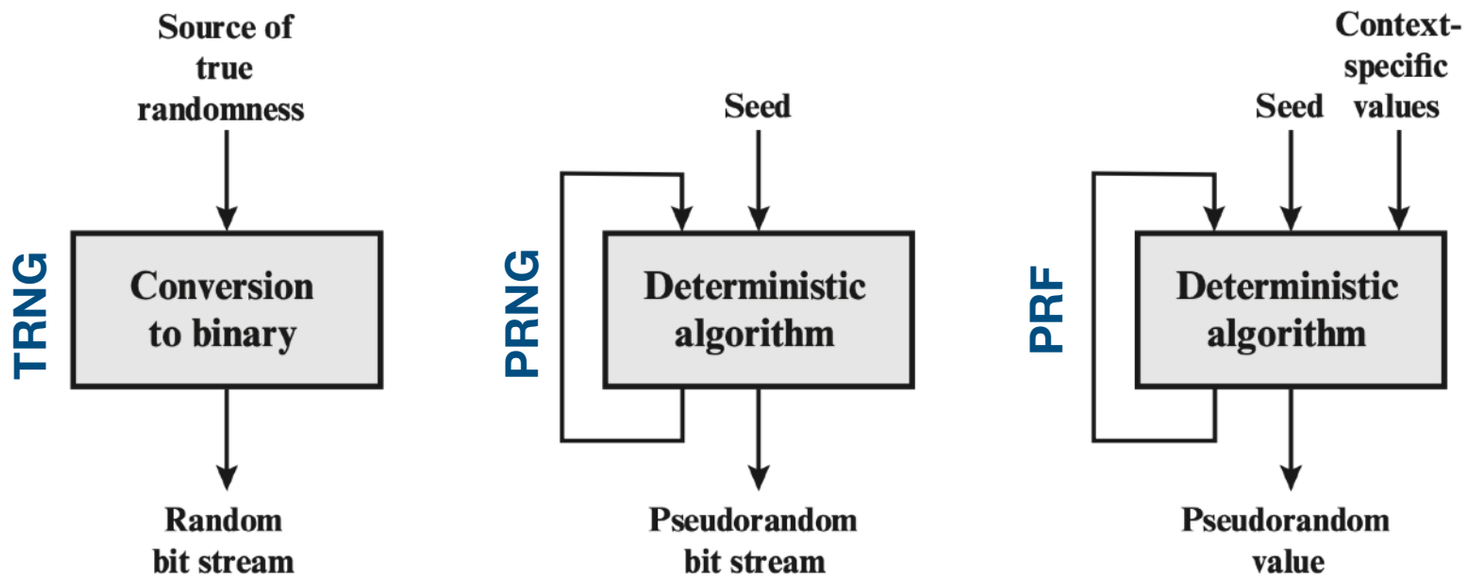
Unpredictability

Each bit should be statically independent of other bits in the sequence. Therefore, unpredictable.

Although true random generators are used in some applications, they have some limitations, such as inefficiency. Thus, it is more common to implement algorithms that generate sequences that appear to be random, *i.e.*, **Pseudorandom numbers**.

Stream Ciphers

Random & Pseudorandom Generators



- **True Random Number Generator (TRNG)**: takes as input a source that is effectively random, *i.e.,* an entropy source (e.g., keystroke timing patterns or instantaneous values of the system clock). It may simply involve the conversion of an analog source to a binary output.

- **Pseudorandom Number Generator (PRNG)**: a *deterministic* algorithm that expands a short, uniform *seed* into an open-ended sequence of bits. It is commonly used as input to a symmetric stream cipher.

The same seed will produce the same result every time the generator is executed.

- **Pseudorandom Function (PRF)**: a deterministic algorithm that expands the seed into a pseudorandom string of bits of some fixed length. Symmetric encryption keys and nonces are examples of their uses.

Stream Ciphers

Pseudorandom Generators

Requirements

Randomness

No single test can determine if a PRNG/PRF generates numbers that have the characteristic of randomness. The best that can be done is to apply a sequence of tests. If the PRGN/PRF exhibits randomness based on multiple trials, then it can be assumed that it satisfies the requirement. The tests should seek for three characteristics:

- **Uniformity:** At any point in the generation of a sequence of pseudorandom bits, the occurrence probability of zero or one is exactly `1/2`.

- **Scalability:** Any test applicable to a sequence must also be applied to subsequences extracted at random. If a sequence is random, then any subsequence should also be random.

- **Consistency:** The behaviour of a generator must be consistent across starting values (i.e., `seeds`).

Stream Ciphers

Pseudorandom Generators

Requirements

Unpredictability

A stream of pseudorandom numbers should exhibit two forms of unpredictability:

- **Forward unpredictability:** If the `seed` is unknown, the next output bit in the sequence should be unpredictable regardless of any knowledge of previous bits in the sequence.

- **Backward unpredictability**: It should not be feasible to determine the `seed` from knowledge of any generated values. No correlation between a `seed` and any value generated from that seed should be evident.

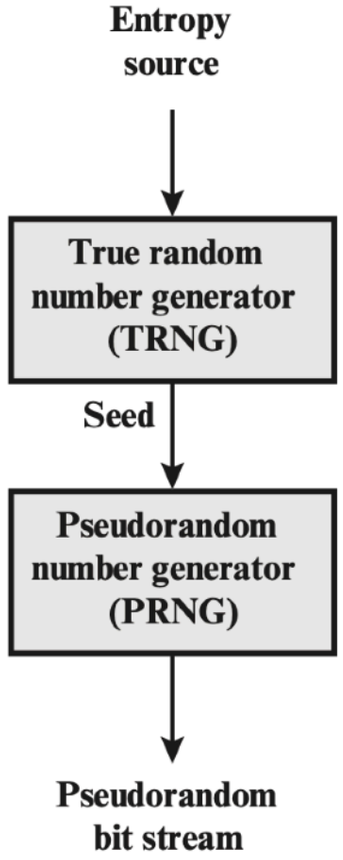
Stream Ciphers

Pseudorandom Generators

SEED Requirements

For cryptographic applications, the `seed` that serves as input must be secure.

- PRNG/PRF are deterministic algorithms. The output can also be determined if an adversary can deduce the `seed`.
- The `seed` must also be unpredictable.
- It is usual to use a TRNG to generate `seeds`.



Stream Ciphers

Pseudorandom Generators

Algorithm design

Purpose-built algorithms

Designed specifically and solely for the purpose of generating pseudorandom bit streams.

- Linear Congruential Generators
- Blum Blum Shut Generator
- For specific stream ciphers, e.g., RC4 and ChaCha20

Algorithms based on existing cryptographic algorithms

Cryptographic algorithms have the effect of randomizing input data (`Confusion` & `Diffusion`)

- Symmetric block ciphers
- Asymmetric ciphers
- Hash functions
- Message Authentication Codes (MAC)

Stream Ciphers

Pseudorandom Generators - Example

Linear Congruential Generator

The algorithm is parameterized with four numbers,

- **m** the modulus **m > 0**
- **a** the multiplier **0 < a < m**
- **c** the increment **0 ≤ c < m**
- **X_0** the starting value/seed **$0 \leq X_0 < m$**

The sequence of random numbers $\{X_n\}$ is obtained via the following iterative equation:

$$X_{n+1} = (aX_n + c) \bmod m$$

- The selection of values for `a`, `c`, and `m` is critical for a good random number generator.
- Three tests can be used to evaluate a generator:
  - The function should be a full-period (it should generate all numbers from `0` to `m-1`).
  - The generated sequence should appear random.
  - The function should implement efficiently with 32-bit arithmetic

$$X_{n+1} = (aX_n) \bmod (2^{31}-1), \text{ for } c=0$$

Stream Ciphers

True Random Number Generators

Entropy sources

A TRNG uses a nondeterministic source to produce randomness.

- Thermal noise
- Sound/Video input
- Disk Drives rotations speed fluctuation

Considerations

- TRNG is impractical for stream ciphers, as the sender would need to securely transmit the ciphertext and keystream as plaintext to the receiver.
- Also, the mechanism used to generate true random numbers may not generate bits at the required rate of the application.

Stream Ciphers

Comparing PRNG and TRNG

	Pseudorandom Generators	True Random Generators
Efficiency	Very efficient	Generally inefficient
Determinism	Deterministic	Nonderterministic
Periodicity	Periodic	Aperiodic

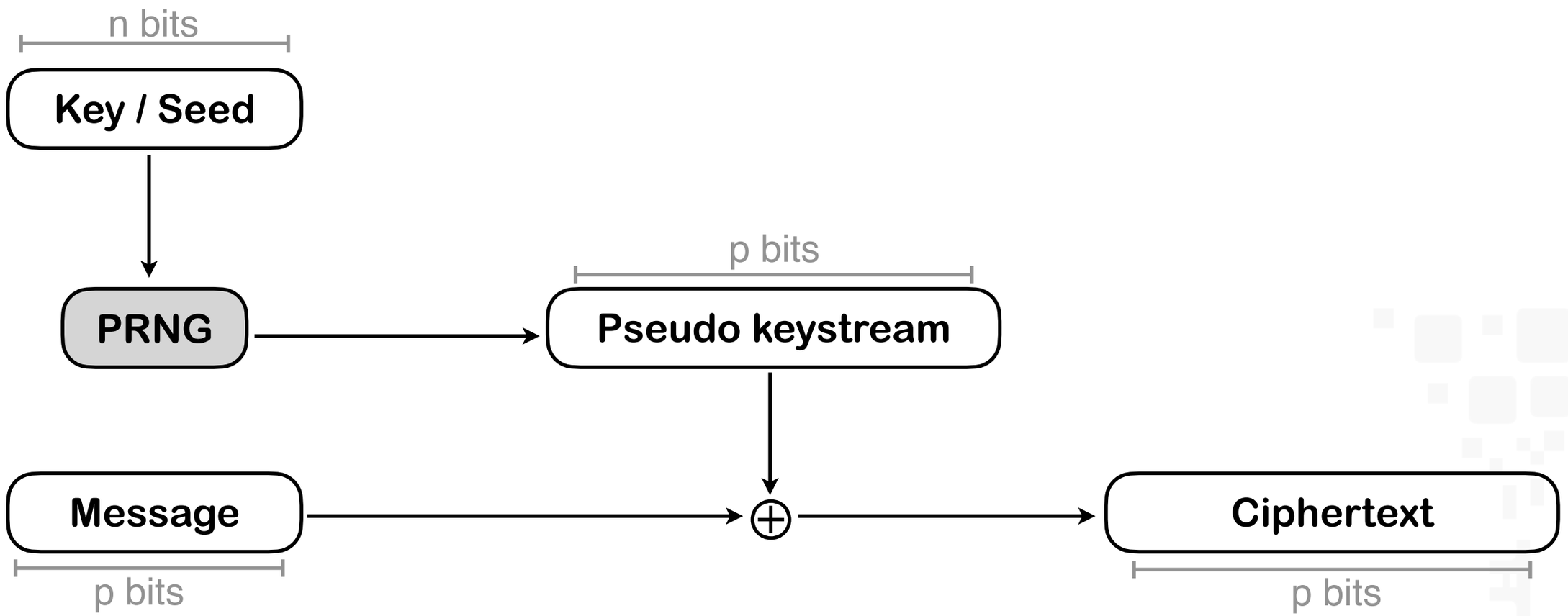
The period of a PRNG is the length of the sequence before it begins to repeat. This is related to the finite internal state and its deterministic nature.

Stream Ciphers

“Pseudo” One-Time Pad

✓ Key as long as the message

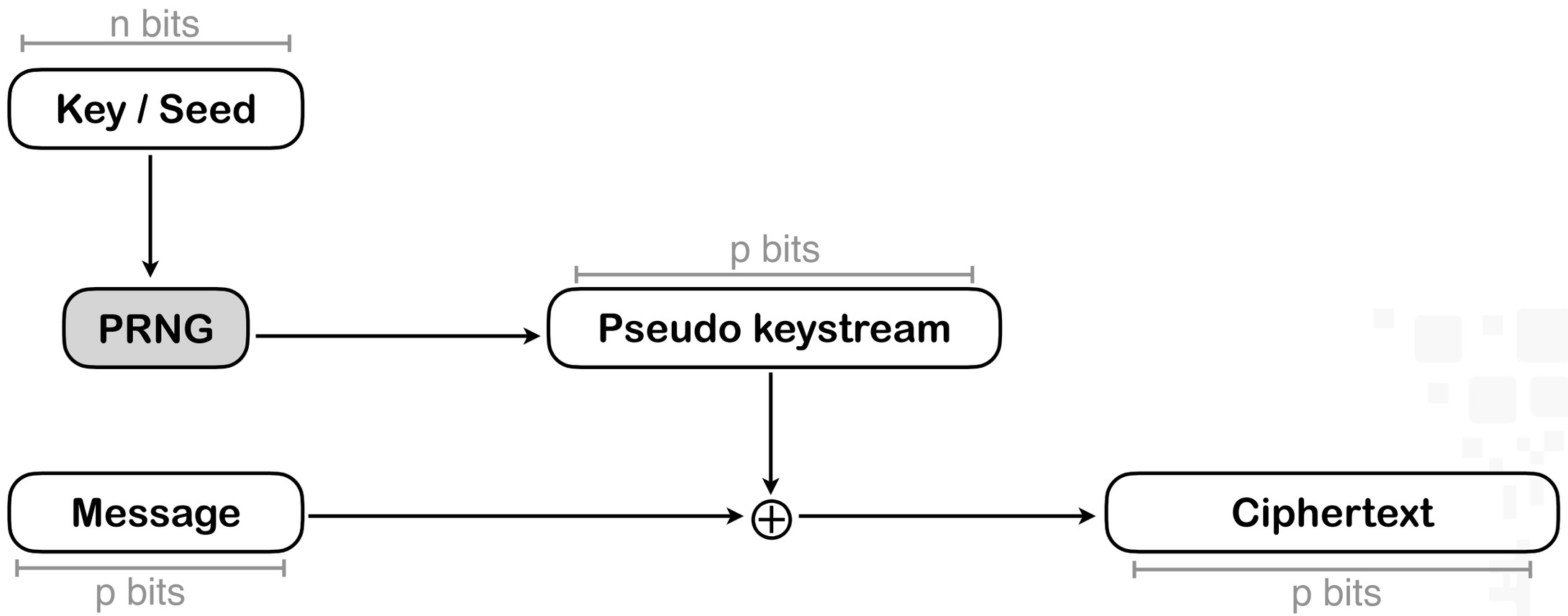
✗ Only secure if each key is used to encrypt once



Stream Ciphers

Hands-on

Develop the Pseudo One-Time Pad encryption scheme by using a pseudorandom generator, such as the os.urandom() function.





Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

University of Minho
School of Engineering